Drone System Final Report

Prepared by
Jay Patel, Manan Patel, Ashton Edakkunathu, Lena Tran
***for use in* CS 440**
at the
University of Illinois Chicago

Spring 2026

Table of Contents

List of Figures

List of Tables

N/a

Project Description

1 Project Overview

Hawkeye is a real-time animal monitoring system that uses a camera module and a Convolutional Neural Network (CNN) to detect and classify animals from live video streams. The system captures video, processes it into frames, and applies a YOLOv26 model to identify animals with bounding boxes and confidence scores, storing detection data in a database for further analysis. By integrating hardware components such as a Raspberry Pi camera with software-based detection and tracking, the system enables continuous wildlife monitoring and supports future extensions such as multi-animal tracking and automated alert systems.

2 Project Domain

The Hawkeye system operates within the domain of computer vision and embedded systems, specifically focusing on real-time object detection for wildlife monitoring. It applies machine learning techniques, particularly Convolutional Neural Networks (CNNs) such as YOLOv26, to process visual data and identify animals from live video streams. This domain requires an understanding of image processing and model inference algorithms on a limited hardware system.

To evaluate the value and correctness of this system, it is important to consider both the accuracy of the detection model and the reliability of the real-time pipeline. Unlike static image classification, this system must continuously process video frames, detect multiple animals, and handle varying environmental conditions such as lighting, motion, and occlusion. Additionally, the integration between hardware, software, and database systems must be verified to ensure that detections are consistently captured, stored, and accessible for further analysis.

3 Relationship to Other Documents

This Implementation Report builds on the work defined by the original group's project documents and focuses on turning those ideas into a functional system. The earlier documentation introduced the concept of a drone-based wildlife monitoring platform, outlining the overall purpose, key features, and expected system behavior. This report reflects how those initial ideas were actually implemented through the development of the Hawkeye system, including the integration of real-time video capture, AI-based animal detection, and cloud data storage.

The Project Requirements Document provided the system expectations, such as accurate real-time detection, handling multiple animals simultaneously, and maintaining reliable data records. These requirements guided the implementation decisions seen in this project, including the use of YOLOv26 for detection, tracking mechanisms to avoid duplicate logging, and database connectivity through Supabase. In addition, the Project Design Document described the system architecture at a high level, which is now reflected in the final structure of the system, where

separate components handle video streaming, model inference, and data management. This report demonstrates how those planned designs were translated into a working and testable solution.

4 Naming Conventions and Definitions

4a Definitions of Key Terms

YOLOv26 (You Only Look Once v26): A convolutional neural network model used for real-time object detection, capable of identifying and classifying animals within video frames.

CNN (Convolutional Neural Network): A type of deep learning model designed for processing and analyzing visual data such as images and video.

Raspberry Pi: A small, low-cost computer used in this project to capture and stream live video from the camera module.

MJPEG Stream: A method of streaming video as a sequence of JPEG images over HTTP, allowing real-time video transmission.

OpenCV: A computer vision library used to process video frames and connect to the live camera stream.

ByteTrack: An object tracking algorithm that assigns unique IDs to detected objects, ensuring the same animal is not counted multiple times.

Supabase: A cloud-based PostgreSQL database used to store detection data such as species, confidence score, and timestamps.

Confidence Score: A value between 0 and 1 that represents how certain the model is about a detection.

4b UML and Other Notation Used in This Document

Figure 1 - User Flow

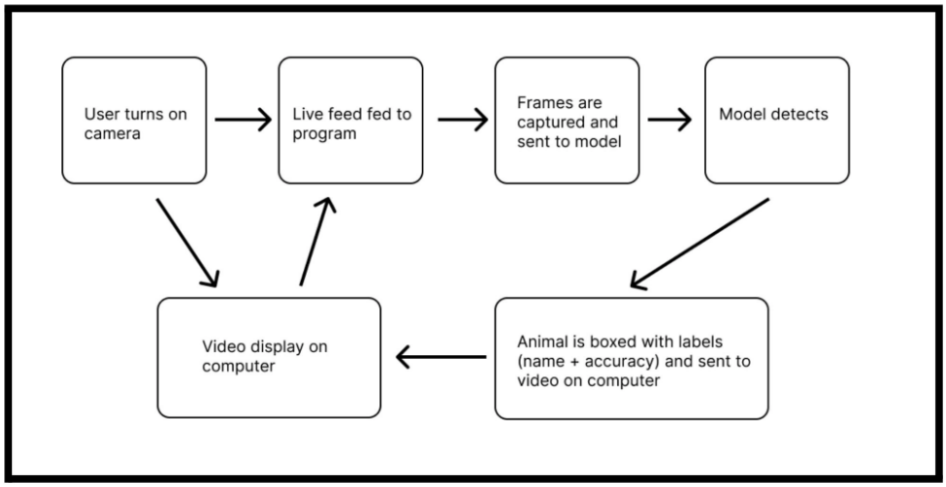

Figure 1 - System User Flow Diagram

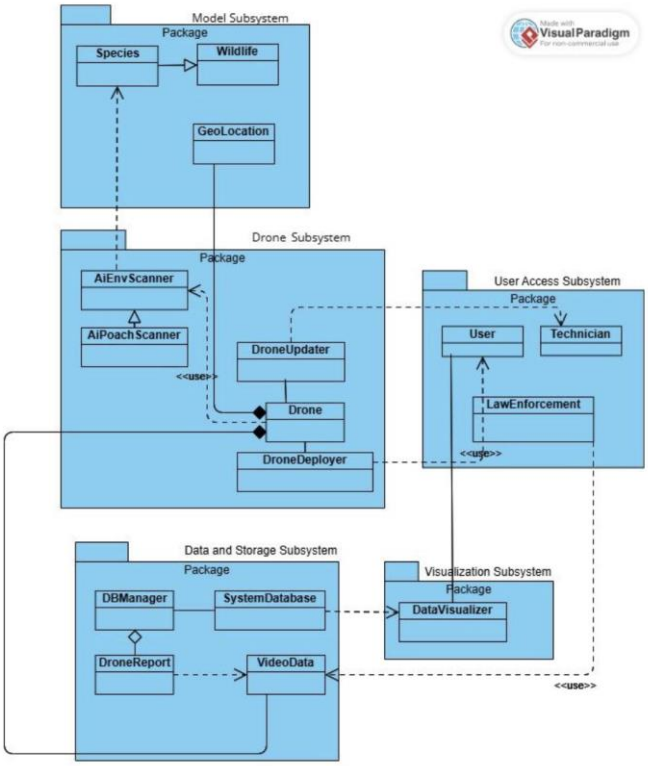

Figure 2 - System Architecture Diagram

Figure 3 - UML Use Case Diagram

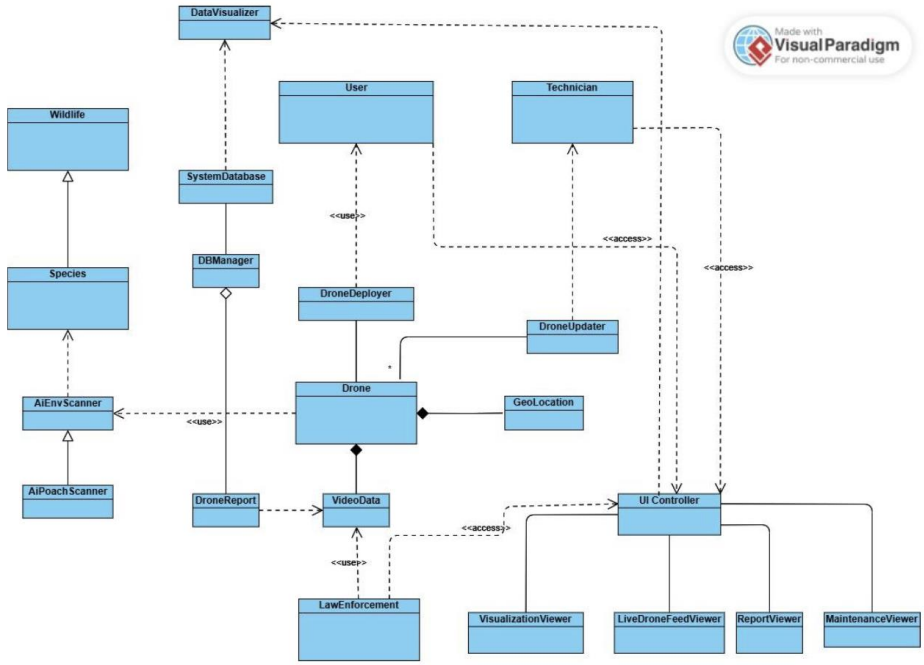

Figure 4 - System Class Diagram

4c Data Dictionary for Any Included Models

The Hawkeye system processes and stores data related to real-time animal detection and monitoring. Data flows from the camera stream through the detection model and into a cloud database, where it is later used for visualization and analysis. The primary data structures are centered around detections, video frames, and system metadata.

Detection Record

Represents a single detected animal event stored in the database.
Detection = ID + Species + Confidence + Timestamp

- **ID:** Integer/String. Unique identifier for each detection entry.
- **Species:** String. The label assigned by the model (e.g., “elephant”, “lion”).
- **Confidence:** Float (0.0 – 1.0). The model’s confidence in the classification.
- **Timestamp:** DateTime. The time at which the detection occurred.

Video Frame

Represents an individual frame captured from the live video stream.
VideoFrame = FrameID + ImageData + Timestamp

- **FrameID:** Integer. Sequential identifier for each frame.
- **ImageData:** JPEG format. Encoded image captured from the camera stream.
- **Timestamp:** DateTime. Time the frame was captured.

Live Video Stream

Represents the continuous feed of frames from the camera module.
LiveStream = SourceIP + Port + Format

- **SourceIP:** String. IP address of the Raspberry Pi camera stream.
- **Port:** Integer. Port used for streaming (e.g., 5000).
- **Format:** String. MJPEG over HTTP.

System Summary Data

Represents aggregated data used for dashboard visualization.
Summary = TotalDetections + SpeciesCount + AverageConfidence

- **TotalDetections:** Integer. Total number of detections recorded.
- **SpeciesCount:** Integer. Number of unique species detected.
- **AverageConfidence:** Float. Average confidence across detections.

II Project Deliverables

Over the course of this semester, Group 4 designed and developed Hawkeye, a drone-mounted wildlife detection and analytics system. The group produced two functional releases, each building on the previous, that together demonstrate a working prototype capable of real-time animal species detection using a convolutional neural network, cloud-based data logging, and a live web dashboard for monitoring and analysis. The following sections document the functionality delivered at each release and how the final prototype compares to the original system design produced by Group 28.

5 First Release

The first release of Hawkeye established the foundational detection pipeline for the system. At this stage, the goal was not end-user functionality but proving that the core software could reliably detect and classify animals in real time before any physical drone hardware was introduced.

The user launched the program by running python detect_animal.py, which initialized the system, displayed a startup message with the program name and version, and attempted to connect to a laptop camera. Upon successful connection, the program streamed continuous live video back to the computer, spliced the video into frames, and passed each frame through a YOLOv26 convolutional neural network. When an animal was detected, the model drew a bounding box around it and displayed the species label and confidence score on screen. Detections that met the confidence threshold were saved to a Supabase cloud database. The user could terminate the session at any time using Ctrl+C.

This release prioritized validating the accuracy and stability of the detection pipeline over any user-facing features. It established the three core components that all subsequent releases would build on: the YOLOv26 CNN model, the real-time video processing pipeline, and the Supabase database integration.

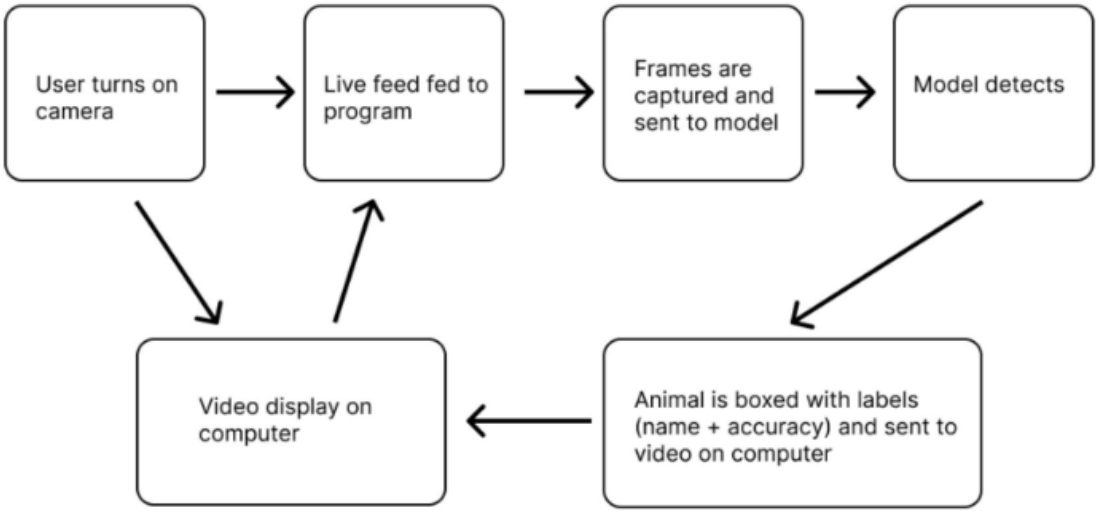

Figure 5 - Flow of Data

6 Second Release

The second release built on the first by introducing real hardware, multi-animal tracking, and autonomous poaching detection. Three major additions distinguished this release from the first.

First, the system transitioned from a laptop camera to a Raspberry Pi camera module to simulate an actual drone-mounted camera. The user connected the Raspberry Pi via USB and launched the system by running python yolo_prediction.py --camera rpi. The system initialized by sequentially connecting to the camera, the Supabase database, and loading the YOLOv26 model, printing a status confirmation for each step. If the camera or database failed to connect, the system printed a descriptive error message and exited gracefully.

Second, multi-animal detection and tracking was introduced. When multiple animals appeared in frame simultaneously, the system processed each one independently, assigning a unique tracking ID to each detected animal. All detections with a confidence score above 80% were saved as separate database entries with the same timestamp. Detections below 80% were displayed on screen but not logged to the database.

Third, autonomous poaching detection was added. The CNN model was trained with annotated examples to recognize potential poaching indicators. When suspicious activity was detected with confidence above 60%, the system drew a red bounding box around the flagged area and printed a POACHING ALERT to the terminal. The detection was automatically queued in a separate poaching_alerts database table storing a snapshot,

timestamp, and confidence score. Manual review functionality was scoped for a future release.

When the user quit the session by pressing Q, the system printed a full session summary including session duration, total animals detected, a species breakdown, and the number of poaching alerts flagged.

7 Comparison with Original Project Design Document

The original project design document produced by Group 28 [5] described an ambitious full-scale drone monitoring platform with multiple autonomous drones, GPS-based navigation, geofencing, a mobile-accessible web interface, and real-time law enforcement alert integration. The Hawkeye prototype successfully realized several of the core design goals outlined in that document while naturally operating at a reduced scope given the constraints of a semester-long implementation project.

The most central requirement of the original design was AI-powered species identification and real-time data transmission to a centralized database. Hawkeye fully achieved this through a custom-trained YOLOv26 convolutional neural network that processed a live camera stream and instantly logged confirmed detections to a Supabase cloud database. The original design specified at least 95% AI identification accuracy and Hawkeye achieved an average confidence score of approximately 94% across all logged detections, closely meeting this target.

The original design also called for a monitoring dashboard where researchers and conservationists could visualize collected data through reports and graphical models. Hawkeye delivered this through a fully functional web dashboard that featured a live camera feed, species breakdown charts, detection trend lines, an activity heatmap, and a searchable detection history with date filtering and CSV export. This directly addressed use cases UC-9 and UC-11 from the original document.

The anti-poaching alert system described in the original design under UC-6 and F-4 called for real-time notification to law enforcement upon detection of poaching activity. Hawkeye made meaningful progress toward this goal by achieving human detection and building an autonomous alert queuing system that flagged suspicious activity and stored it in a separate database table for review. Full real-time authority notification was scoped for a future release.

Several features from the original design were not implemented in the prototype due to scope and resource constraints. The original design envisioned a fleet of multiple autonomous drones with GPS-based navigation and geofencing across large geographic areas while Hawkeye simulates a single drone using a Raspberry Pi camera module. Features such as mobile application support, GIS integration, role-based user access control, and waiting room features including VR/AR visualization and wearable tag integration were not included in the prototype but remain viable directions for future development.

Overall, Hawkeye demonstrates that the core technical vision described by Group 28 is achievable. The prototype successfully validated the most critical components of the system including real-time AI detection, cloud data storage, and data visualization, and established a strong foundation for the full system described in the original design document.

III Testing

8 Items to be Tested

The Hawkeye system was tested across its core components, including video capture, real-time detection, data storage, and visualization. Testing focused on ensuring that each part of the pipeline functions correctly both independently and as part of the full system.

Items:

Camera Initialization (Raspberry Pi / Laptop Camera)
Live Video Stream (MJPEG feed)
Frame Capture and Processing
YOLOv8 Model Loading and Inference
Animal Detection Accuracy (bounding boxes + labels)
Confidence Threshold Filtering (e.g., only save detections above threshold)
Multi-Animal Detection Handling
Object Tracking / Duplicate Prevention
Database Connection (Supabase)
Detection Logging to Database

9 Test Specifications

ID# 1 – Animal Detection Test
Description: System detects and classifies animals from live video feed.
Items covered by this test: YOLOv8 Model, Frame Processing
Requirements addressed by this test: F-1, D-2
Environmental needs: Laptop or Raspberry Pi camera, Python environment, YOLOv8 model
Intercase Dependencies: NA
Test Procedures:
- Run detection script
- Start live camera feed
- Observe detection output on screen
Input Specification: Live video containing animals
Output Specifications: Bounding boxes with species labels and confidence scores
Pass/Fail Criteria: Animals are correctly detected and labeled with reasonable confidence

ID# 2 – Real-Time Processing Test
Description: System processes frames continuously with minimal delay.
Items covered by this test: Video Stream, Frame Processing
Requirements addressed by this test: F-2
Environmental needs: Camera stream, OpenCV, detection script
Intercase Dependencies: Test 1
Test Procedures:
- Run detection system
- Observe processing speed
- Check for lag or freezing
Input Specification: Continuous video feed
Output Specifications: Smooth video with near real-time detection updates
Pass/Fail Criteria: System runs without noticeable delay or crashes

ID# 3 – Database Logging Test
Description: System stores detected animals into the database.
Items covered by this test: Database Connection, Detection Logging
Requirements addressed by this test: F-3, D-2
Environmental needs: Supabase database, valid credentials
Intercase Dependencies: Test 1
Test Procedures:
- Run detection system
- Trigger detection above threshold
- Check database for new entry
Input Specification: Detection with confidence above threshold
Output Specifications: New record with species, confidence, timestamp
Pass/Fail Criteria: Detection is correctly stored in database

ID# 4 – Live Video Display Test
Description: System displays live video feed with detection overlays.
Items covered by this test: Video Display, Bounding Box Rendering
Requirements addressed by this test: F-5
Environmental needs: Camera stream, display window
Intercase Dependencies: Test 1
Test Procedures:
Start system
- Observe video output
- Input Specification: Live video feed
- Output Specifications: Video with bounding boxes and labels
Pass/Fail Criteria: Video displays correctly with overlays

ID# 5 – System Stability Test
Description: System runs continuously without failure over time.
Items covered by this test: System Stability, Continuous Processing
Requirements addressed by this test: Reliability Requirements

Environmental needs: Full system setup
Intercase Dependencies: Tests 1–5
Test Procedures:
- Run system for 10–15 minutes
- Monitor performance
- Input Specification: Continuous video stream
Output Specifications: System runs without interruption
Pass/Fail Criteria: No crashes or major slowdowns occur

10 Test Results

ID# 1 – Animal Detection Test
Date(s) of Execution: 2026-04-25
Staff conducting tests: Jay Patel
Expected Results: Animals are detected with correct labels and confidence scores
Actual Results: Animals were detected correctly with bounding boxes and labels; minor misclassifications occurred at low confidence levels
Test Status: Pass (Overall accurate detection, minor edge case errors)

ID# 2 – Real-Time Processing Test
Date(s) of Execution: 2026-04-25
Staff conducting tests: Manan Patel
Expected Results: System processes video frames in real-time with minimal delay
Actual Results: System processed frames with slight delay (~1 second), no freezing observed
Test Status: Pass (Acceptable real-time performance)

ID# 3 – Database Logging Test
Date(s) of Execution: 2026-04-25
Staff conducting tests: Manan Patel
Expected Results: Detections above threshold are stored in database with correct fields
Actual Results: Records successfully inserted into Supabase with species, confidence, and timestamp; no data loss observed
Test Status: Pass

ID# 5 – Live Video Display Test
Date(s) of Execution: 2026-04-25
Staff conducting tests: Ashton Edakkunathu
Expected Results: Live video displays with bounding boxes and labels
Actual Results: Video feed displayed correctly with real-time overlays; no crashes or rendering issues
Test Status: Pass

ID# 6 – System Stability Test
Date(s) of Execution: 2026-04-25

Staff conducting tests: Lena Tran
Expected Results: System runs continuously without crashing or major slowdowns
Actual Results: System ran for ~15 minutes without crashes; slight performance drop over time but remained functional
Test Status: Pass (Minor performance degradation over time)

11 Regression Testing

No repeated tests

IV Inspection

12 Items to be Inspected

12a CNN Prediction Model

12b CNN Training

12c Camera Initialization

12d MPEG feed

12e Multi Animal Detection

12f Persistent tracking

12g Confidence Filtering

12h Connection to Database

13 Inspection Procedures

13a Weekly meetings, usually at the end of week, to discuss what was accomplished and to set goals for the next week.

13b Always plan in pairs, to allow for ideas to bounce off each other.

13c Always make sure at least 2 people in the team understand the code before pushing

13d Always test edge cases.

13e Ensure to test the code via execution.

14 Inspection Results

13d Lena Tran: Tested UI, Supabase. Discovered the repeated data logging and failure to detect multiple of the same animal. Resolved by adding logic to persist the CNN's knowledge of each animal and to only log newly IDed animals.

13e Jay Patel: Tested the Data analytics and partially the hardware. No flaws were found and no rectification was needed.

13e Manan Patel: Tested the hardware. Found out that hardware could not connect to UIC Wi-fI due to excess security and hotspots resulted in 100% packet loss of data. No resolution could be found for this, noted as a hardware/security issue.

13e Ashton Edakkunnathu: Tested CNN. Discovered catastrophic forgetting in the model during retrained. Resolved by retraining the model with older data as well as new data.

V Recommendations and Conclusions

The core components of the drone-based anti-poaching system have been done and ready to move forward to the next phase of development. The Raspberry Pi hardware with the integrated camera module is functional and working well for the current stage of the project, the mini UI dashboard is operational, the Supabase database is successfully storing and managing detection data, and the CNN model along with its data visualizations are at an acceptable level. Human detection has been introduced, confirming that the system can identify people within the camera's field of view. With these components in a good state, the next step is to work toward poaching behavior detection, where the model would need to go beyond just spotting people and start identifying suspicious or threatening activity in wildlife areas. If the current setup using Supabase and Raspberry Pi continues to work well, it is recommended to eventually move toward larger and more powerful cloud platforms such as AWS, GCP, or Azure, along with stronger hardware to support better performance and flight. It is also recommended to start collecting real world field data in actual wildlife environments, as this will help identify areas where the system needs improvement since all work this semester has been on images and videos.

VI Project Issues

15 Open Issues

Currently, there are a few open issues that apply to our system. One main issue is

connectivity, since network connections are not always stable. Throughout the semester we mainly used the Raspberry Pi at home where the Wi-Fi was easiest to connect to, while UIC Wi-Fi and mobile hotspots showed inconsistent results. As the project moves forward, having a stable wireless connection to stream video and run detections in the field will be necessary, which is why the hardware and infrastructure upgrades mentioned in Part V should be considered. Another issue worth noting is model performance. While the current CNN model does deliver high confidence results on certain images, it was trained with limited hardware and may not be reliable enough for real field use. This is not as urgent as the connectivity issue, but improving the model with better training data and more powerful hardware will be important as the project continues to grow. An additional issue to consider is data privacy and legal regulations around drone usage. Depending on the location, certain areas may have flight restrictions or rules around aerial surveillance that could limit where and how the system can be deployed, which would need to be looked into before any real field testing takes place.

16 Waiting Room

There are several features and ideas that were considered during the development of this project but could not be included in the current version due to time and hardware limitations. The most immediate feature planned for the next release is poaching behavior detection, which would take the system beyond identifying people and toward recognizing suspicious or threatening activity in wildlife areas. Automated drone flight paths are also planned for a future release, allowing the drone to patrol a set area on its own without needing to be manually controlled. Further down the line, multi-drone support would be a valuable addition, enabling multiple drones to work together to cover larger wildlife areas more efficiently. Mobile alert notifications are another future consideration, where rangers or users would receive real time alerts on their phones when a detection is made. Finally, night vision and thermal camera support would be explored in a later release to improve detection reliability in low light conditions and during nighttime hours when poaching activity is more likely to occur.

17 Ideas for Solutions

Several ideas have come up throughout the development of this project that could be worth exploring in future versions. For improving the detection model, looking into YOLOv26 as an alternative architecture could be beneficial since it is well known for performing well in real time object detection scenarios. For handling data at a larger scale, migrating to a cloud platform such as AWS or Google Cloud would allow for better storage, processing, and overall system reliability compared to the current Supabase setup. Improving the camera feed processing could also be explored through deeper use of OpenCV(since Pi kept crashing from pip installs), which offers a wide range of tools for image handling that could help with detection accuracy. For the mobile alert feature mentioned in the waiting

room, a service like Firebase could be a straightforward way to push real time notifications to users when a detection is made. Finally, when automated drone flight paths become a priority, look into existing drone software and flight control libraries which can give a good starting point rather than building that functionality from scratch.

18 Project Retrospective

Overall this project was a valuable experience for the team, especially since working with a machine learning model and building a full pipeline around it was a first for everyone involved. A lot was learned throughout the semester and the project ended in a solid place with multiple working components. Things like the database, UI dashboard, and visualizations came together smoothly and did not cause many issues. The main roadblocks encountered were the Raspberry Pi connectivity issues mentioned earlier, which took time away from other areas of development. One thing that could have used more attention was the actual drone flight aspect, as the team simulated camera input by just picking up Pi and pointing the camera at image/video rather than having a drone actively flying, so getting a real drone in the air is something that would need more focus going forward. That said, the decision to prioritize functionality first was reasonable given the time and resources available. The project reached a decent stopping point with all the core pieces working together, and realistically the next steps are building out the actual drone setup and testing the system with real world conditions and live data, which would bring the project much closer to being a true minimum viable product.

VII Glossary

CNN (Convolutional Neural Network) - A type of artificial intelligence model commonly used for image recognition and computer vision tasks. CNNs analyze visual patterns in images or video frames to identify objects, shapes, or features automatically.

Confidence Score - A numerical value produced by an AI model that represents how certain the system is about a prediction or detection. Higher confidence scores indicate greater certainty in the result.

Hawkeye - A computer vision and tracking system used to monitor wildlife in real time. In this project, Hawkeye refers to the system responsible for detecting, tracking, and processing visual data from cameras.

MJPEG (Motion JPEG) - A video compression format where each video frame is stored as an individual JPEG image. MJPEG is commonly used for live video streaming because it is simple and provides low-latency transmission.

Raspberry Pi - A small, affordable single-board computer often used in embedded systems, robotics, and IoT applications. It can run software, process sensor data, and interface with cameras and other hardware components.

Supabase - An open-source backend platform that provides database management, authentication, APIs, and cloud storage services. It is often used as an alternative to Firebase for full-stack application development.

UML (Unified Modeling Language) - A standardized visual language used to design and document software systems. UML diagrams help developers represent system architecture, workflows, object relationships, and processes.

YOLOv26 (You Only Look Once Version 26) - A real-time object detection model in the YOLO family of computer vision algorithms. It is designed to quickly identify and locate objects within images or video streams with high accuracy.

VIII References / Bibliography

[1] Robertson and Robertson, Mastering the Requirements Process.

[2] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, Ninth ed., Wiley, 2013.

[3] J. Bell, "Underwater Archaeological Survey Report Template: A Sample Document for Generating Consistent Professional Reports," Underwater Archaeological Society of Chicago, Chicago, 2012.

[4] M. Fowler, UML Distilled, Third Edition, Boston: Pearson Education, 2004.

[5] Group 28 (L. Cruz, O. Neal, E. Aguayo, T. Kang), "Drone System to Monitor Animals," CS 440 Project Design Document, University of Illinois Chicago, Spring 2025.

IX Index